

Instalación y configuración de PostgreSQL 17 en Debian 13

1. Objetivo

En esta práctica se instala PostgreSQL en un equipo con Debian 13. Después se configura el servidor para aceptar conexiones TCP/IP desde la red local y se crea una base de datos con un usuario propio para realizar las pruebas.

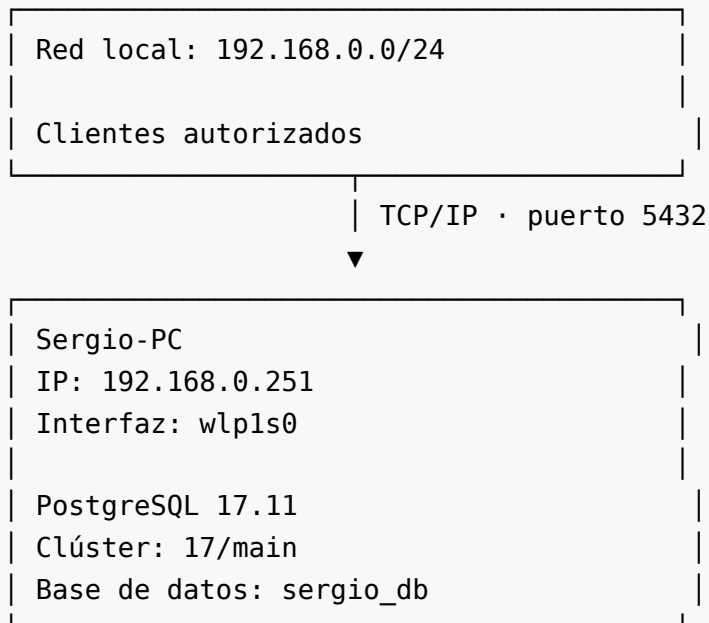
La conexión se verifica usando la dirección IP de la red local del equipo:

```
IP del servidor PostgreSQL: 192.168.0.251
Puerto PostgreSQL:        5432
Base de datos de pruebas:  sergio_db
Usuario de pruebas:        sergio
```

2. Entorno utilizado

Alumno	Sergio Mesa
Fecha	7 de octubre de 2026
Equipo servidor	Sergio-PC
Sistema operativo	Debian 13
Servidor de base de datos	PostgreSQL 17.11
Clúster	17/main
Puerto	5432
Directorio de datos	/var/lib/postgresql/17/main
Fichero de log	/var/log/postgresql/postgresql-17-main.log
Interfaz de red local	wlp1s0
IP de la red local	192.168.0.251/24
Red permitida	192.168.0.0/24

3. Esquema de funcionamiento



4. Instalación de PostgreSQL

Se actualiza el índice de paquetes de Debian y se instalan PostgreSQL y el paquete de extensiones adicionales:

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
```

- `postgresql` instala el servidor, el cliente `psql` y el clúster principal de PostgreSQL.
- `postgresql-contrib` instala extensiones adicionales que pueden ser útiles en futuras prácticas.

Después de la instalación se comprueba que el servicio está configurado para iniciarse automáticamente y se encuentra activo:

```
sudo systemctl is-enabled postgresql
sudo systemctl is-active postgresql
sudo systemctl status postgresql --no-pager
```

Resultado obtenido:

```
enabled
active
```

En Debian, `postgresql.service` puede aparecer como `active (exited)` . Esto es normal: esa unidad principal se encarga de gestionar los clústeres de PostgreSQL. El estado real del clúster se comprueba con `pg_lsclusters` .

Se comprueba la versión instalada:

```
psql --version
```

Resultado obtenido:

```
psql (PostgreSQL) 17.11 (Debian 17.11-0+deb13u1)
```

Se comprueba el clúster creado por Debian:

```
pg_lsclusters
```

Resultado obtenido:

Ver	Cluster	Port	Status	Owner	Data directory	Log file
17	main	5432	online	postgres	/var/lib/postgresql/17/main	/var/log/postgresql/postgresql-17-main.log

Este resultado confirma que el clúster `17/main` está en estado `online` , pertenece al usuario del sistema `postgres` y escucha en el puerto `5432` .

5. Comprobación local de PostgreSQL

Se accede a PostgreSQL usando el usuario administrador del sistema `postgres` :

```
sudo -u postgres psql
```

Dentro de la consola de PostgreSQL se ejecuta la consulta siguiente:

```
SELECT version();
```

Resultado obtenido:

```
PostgreSQL 17.11 (Debian 17.11-0+deb13u1) on x86_64-pc-linux-gnu,  
compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
```

También se puede comprobar la base de datos, el usuario y el puerto con esta consulta:

```
SELECT  
  current_database() AS base_datos,  
  current_user AS usuario,  
  inet_server_addr() AS direccion_servidor,  
  inet_server_port() AS puerto;
```

Para salir de `psql` se utiliza:

```
\q
```

6. Identificación de la red local

Antes de configurar el acceso remoto se revisan las direcciones de red del equipo:

```
hostname -I  
ip -br a
```

La interfaz conectada a la red local es `wlp1s0` y tiene esta dirección:

```
192.168.0.251/24
```

Por tanto, la red local utilizada es:

```
192.168.0.0/24
```

7. Copia de seguridad de la configuración

Antes de modificar la configuración se crean copias de seguridad de los ficheros principales:

```
sudo cp /etc/postgresql/17/main/postgresql.conf \
/etc/postgresql/17/main/postgresql.conf.bak
```

```
sudo cp /etc/postgresql/17/main/pg_hba.conf \
/etc/postgresql/17/main/pg_hba.conf.bak
```

Los ficheros de configuración utilizados son:

```
/etc/postgresql/17/main/postgresql.conf
/etc/postgresql/17/main/pg_hba.conf
```

- `postgresql.conf` contiene la configuración general del servidor, incluyendo las direcciones IP en las que escucha.
- `pg_hba.conf` controla qué clientes pueden conectarse, desde qué redes y con qué método de autenticación.

8. Configuración de escucha TCP/IP

Se abre el fichero principal de configuración:

```
sudo nano /etc/postgresql/17/main/postgresql.conf
```

En el fichero se añade o se modifica la siguiente línea:

```
listen_addresses = '127.0.0.1,192.168.0.251'
```

Con esta configuración PostgreSQL escucha en dos direcciones:

Dirección	Función
127.0.0.1	Permite conexiones TCP/IP desde el propio equipo
192.168.0.251	Permite conexiones desde la red local a través de la interfaz Wi-Fi

No se utiliza `0.0.0.0`, ya que eso expondría PostgreSQL en todas las interfaces de red del equipo, incluidas las redes virtuales y Tailscale.

9. Configuración de acceso en `pg_hba.conf`

Se abre el fichero de reglas de autenticación:

```
sudo nano /etc/postgresql/17/main/pg_hba.conf
```

Al final del fichero se añade la siguiente regla:

```
host    all    all    192.168.0.0/24    scram-sha-256
```

La regla se interpreta de la siguiente forma:

Campo	Valor	Significado
Tipo	host	La regla se aplica a conexiones TCP/IP
Base de datos	all	Permite acceder a cualquier base de datos
Usuario	all	Permite acceder a cualquier rol con permisos válidos
Red	192.168.0.0/24	Solo admite clientes de la red local
Método	scram-sha-256	Solicita una contraseña usando autenticación segura

Las reglas originales ya permitían conexiones locales mediante `127.0.0.1` y `::1`. La nueva regla añade acceso desde equipos de la misma red local.

10. Reinicio y comprobación del puerto

Después de modificar `postgresql.conf` es necesario reiniciar PostgreSQL:

```
sudo systemctl restart postgresql
```

Se comprueba que el servicio sigue activo:

```
sudo systemctl is-active postgresql
```

Resultado obtenido:

```
active
```

Se comprueba que PostgreSQL escucha tanto en localhost como en la IP de la red local:

```
sudo ss -ltnp | grep ':5432'
```

Resultado obtenido:

```
LISTEN 0 200 127.0.0.1:5432          0.0.0.0:* users:(("postgres",pid=27421,fd=6))
LISTEN 0 200 192.168.0.251:5432       0.0.0.0:* users:(("postgres",pid=27421,fd=7))
```

Este resultado confirma que PostgreSQL acepta conexiones TCP/IP por el puerto `5432` en la interfaz local y en la interfaz de red `192.168.0.251`.

11. Creación de usuario y base de datos

No se utiliza el usuario administrador `postgres` para las conexiones de prueba. Se crea un rol propio y una base de datos asociada.

Se abre `psql` como administrador:

```
sudo -u postgres psql
```

Dentro de PostgreSQL se ejecutan los siguientes comandos:

```
CREATE ROLE sergio LOGIN PASSWORD 'CONTRASEÑA';
CREATE DATABASE sergio_db OWNER sergio;
```

Para salir:

```
\q
```

El rol `sergio` tiene permiso de inicio de sesión y es propietario de la base de datos `sergio_db`.

La contraseña no se muestra ni se guarda en este documento.

12. Prueba de conexión por la IP de red

La conexión se prueba usando la IP de la red local, no `localhost`. De este modo se comprueba que PostgreSQL acepta conexiones TCP/IP en la dirección configurada.

```
psql -h 192.168.0.251 -p 5432 -U sergio -d sergio_db
```

Después de introducir la contraseña del usuario `sergio`, se establece una conexión SSL con PostgreSQL:

```
Conexión SSL (protocolo: TLSv1.3, cifrado: TLS_AES_256_GCM_SHA384,
compresión: desactivado, ALPN: postgresql)
```

Dentro de `psql` se ejecuta la consulta de validación:

```
SELECT
  current_database() AS base_datos,
  current_user AS usuario,
  inet_client_addr() AS cliente,
  inet_server_addr() AS servidor,
  inet_server_port() AS puerto;
```

Resultado obtenido:

```
base_datos | usuario | cliente | servidor | puerto
-----+-----+-----+-----+-----
sergio_db | sergio | 192.168.0.251 | 192.168.0.251 | 5432
(1 fila)
```

La consulta confirma que:

- La conexión se ha realizado mediante TCP/IP.
- PostgreSQL recibe la conexión en `192.168.0.251`.
- El puerto utilizado es `5432`.
- La autenticación se realiza con el usuario `sergio`.
- La base de datos utilizada es `sergio_db`.

13. Comprobación final

Como comprobación final se revisa de nuevo el puerto de escucha:

```
sudo ss -ltnp | grep ':5432'
```

Resultado obtenido:

```
LISTEN 0 200 127.0.0.1:5432      0.0.0.0:* users:(("postgres",pid=27421,fd=6))
LISTEN 0 200 192.168.0.251:5432 0.0.0.0:* users:(("postgres",pid=27421,fd=7))
```

PostgreSQL queda instalado, configurado para el inicio automático y preparado para aceptar conexiones desde la red local `192.168.0.0/24` por el puerto `5432` .

14. Problemas frecuentes

PostgreSQL solo escucha en `127.0.0.1`

Comprobar que `postgresql.conf` contiene:

```
listen_addresses = '127.0.0.1,192.168.0.251'
```

Después reiniciar el servicio:

```
sudo systemctl restart postgresql
```

Y comprobar el puerto:

```
sudo ss -ltnp | grep ':5432'
```

Error de autenticación desde otra máquina

Comprobar que `pg_hba.conf` incluye la regla:

```
host      all      all      192.168.0.0/24      scram-sha-256
```

Después se puede reiniciar PostgreSQL:

```
sudo systemctl restart postgresql
```

También se puede recargar la configuración sin reiniciar la instancia:

```
sudo systemctl reload postgresql
```

No se puede conectar al puerto 5432

Comprobar el estado del clúster:

```
pg_lsclusters
```

Comprobar el servicio:

```
sudo systemctl status postgresql --no-pager
```

Comprobar la escucha de red:

```
sudo ss -ltnp | grep ':5432'
```

Si se utiliza UFW, permitir conexiones solamente desde la red local:

```
sudo ufw allow from 192.168.0.0/24 to any port 5432 proto tcp
```

15. Conclusión

Se ha instalado PostgreSQL 17.11 en Debian 13. El clúster `17/main` se encuentra activo, se inicia automáticamente y utiliza el puerto `5432`.

El servidor se ha configurado para escuchar en `127.0.0.1` y en la dirección de red local `192.168.0.251`. Además, `pg_hba.conf` permite conexiones desde la red `192.168.0.0/24` utilizando autenticación `scram-sha-256`.

Se creó el usuario `sergio` y la base de datos ``sergio_db``.